

DriftWatch — ML Model Drift Detection Pipeline

■■ ENDGAME TIER — MLOps Engineering

"Automated Data & Concept Drift Monitoring for Production ML Systems"

■■ Endgame Project | Domain: MLOps + Statistical Monitoring | Difficulty: Advanced | Timeline: 10 Weeks

■ Project Overview

Description	An automated production-grade monitoring pipeline that detects data drift and concept drift in deployed ML models using statistical tests (KS-test, PSI, Chi-Square, Jensen-Shannon Divergence). Triggers intelligent retraining alerts, logs feature-level degradation timelines, and visualises model health via a live Grafana-style Streamlit dashboard.
Target Audience	MLOps engineers, senior ML engineers, production ML system builders
Domain	MLOps, Statistical Process Control, Model Monitoring
Deliverable	Automated drift detection pipeline + alerting system + live monitoring dashboard + retraining trigger logic
Why It Stands Out	Most portfolios show model training. DriftWatch shows you understand what happens AFTER deployment — the production ML lifecycle that separates juniors from seniors.

■■ Tech Stack

Language	Python 3.10+
Drift Detection	Evidently AI, scipy (KS-test, Chi-Square), custom PSI
Statistical Tests	KS-test, PSI (Population Stability Index), JSD, Chi-Square

ML Framework	scikit-learn, XGBoost (monitored models)
Pipeline Orchestration	Apache Airflow (or Prefect as lighter alternative)
Storage	PostgreSQL (drift logs), MinIO or local FS (model/data snapshots)
Dashboard	Streamlit (Grafana-style layout) + Plotly
Alerting	SMTP email alerts / Slack webhook notifications
Containerisation	Docker + Docker Compose
Dev Tools	Jupyter, VSCode, Git, MLflow for model versioning

■ MVP Features (Phase 1)

Baseline Model & Reference Data Setup

- ◆ Train a production baseline model (classifier or regressor) on reference dataset
- ◆ Save model, training data schema, and feature statistics as the reference snapshot
- ◆ Define baseline feature distributions (mean, std, quantiles, value frequencies)
- ◆ Version baseline with MLflow: model artifacts + metadata + dataset hash

Data Drift Detection Engine

- ◆ KS-test (Kolmogorov-Smirnov) for continuous feature drift detection
- ◆ Chi-Square test for categorical feature distribution shifts
- ◆ PSI (Population Stability Index): PSI < 0.1 stable, 0.1-0.25 warning, > 0.25 critical
- ◆ Jensen-Shannon Divergence for probability distribution comparison
- ◆ Per-feature drift scores aggregated into an overall drift score

Concept Drift Detection Engine

- ◆ Monitor model performance metrics over sliding time windows (7-day, 30-day)
- ◆ Track accuracy, F1, AUC-ROC decay curves on incoming labelled batches
- ◆ Page-Hinkley test for sequential change-point detection in error rates
- ◆ ADWIN (Adaptive Windowing) for detecting abrupt concept drift

Alerting & Retraining Trigger

- ◆ Configurable alert thresholds: WARNING (PSI > 0.1), CRITICAL (PSI > 0.25)
- ◆ Slack webhook + email alert on drift detection with feature-level breakdown
- ◆ Automatic retraining trigger: flag model for retraining when drift is critical
- ◆ Alert deduplication: no repeat alerts within 24h for the same feature

Airflow / Prefect Pipeline

- ◆ Scheduled DAG: run drift checks daily on new incoming batch data
- ◆ DAG tasks: ingest new data → compute drift scores → log to DB → alert if threshold breached
- ◆ Retry logic and failure notifications for pipeline task failures
- ◆ Drift report generated as JSON artifact per run

Monitoring Dashboard

- ◆ Feature drift timeline chart: per-feature PSI/KS score over time
- ◆ Model performance decay chart: accuracy/F1 trend over rolling windows
- ◆ Traffic volume monitor: incoming prediction request counts over time
- ◆ Alert history log: past drift events and their severity

■ Development Roadmap

Week 1 Project Setup & Baseline Model	■ Set up Python environment, Docker, PostgreSQL via Docker Compose ■ Select and train a baseline classification model (e.g. income prediction on Adult dataset) ■ Log baseline model to MLflow with all metadata and dataset hash ■ Save reference feature statistics (mean, std, quantiles, value counts) as JSON
Week 2 Synthetic Drift Simulation	■ Build a drift simulator: tool to artificially inject drift into incoming data batches ■ Simulate 3 types of drift: covariate shift, prior probability shift, concept drift ■ Create 10 synthetic incoming data batches with varying drift levels (none/mild/severe) ■ Validate drift is visually detectable before building automated detection
Week 3 Data Drift Detection — Statistical Tests	■ Implement KS-test for all continuous features (compare reference vs incoming) ■ Implement Chi-Square test for all categorical features ■ Implement PSI computation with configurable bin strategy ■ Implement Jensen-Shannon Divergence computation ■ Aggregate per-feature scores into an overall drift health score
Week 4 Concept Drift Detection	■ Implement sliding-window performance monitoring (collect predictions + labels) ■ Compute accuracy, F1, AUC-ROC on 7-day and 30-day rolling windows ■ Implement Page-Hinkley sequential change-point test on error rate ■ Integrate Evidently AI as a secondary drift detection framework for validation
Week 5 Drift Logging to PostgreSQL	■ Design drift_events table schema (feature, test_type, score, threshold, severity, timestamp) ■ Design model_performance_log table (window_start, window_end, accuracy, f1, auc) ■ Build SQLAlchemy ORM models for both tables ■ Write drift scores + performance metrics to DB after each detection run
Week 6 Alerting System	■ Build threshold-based alert classifier (OK / WARNING / CRITICAL) ■ Implement Slack webhook alert with feature breakdown message ■ Implement SMTP email alert with HTML drift summary report ■ Add alert deduplication logic (cooldown period per feature per severity) ■ Log all alerts to alert_history table in PostgreSQL
Week 7 Airflow Pipeline Orchestration	■ Set up Apache Airflow (or Prefect) via Docker ■ Build daily drift monitoring DAG with 4 tasks: ingest → detect → log → alert ■ Add DAG retry logic (3 retries with exponential backoff) ■ Add pipeline failure email notification ■ Test end-to-end pipeline with synthetic drift batches

**Week 8 Streamlit
Monitoring
Dashboard**

- Build feature drift timeline chart: PSI/KS score per feature over 30 days
- Build model performance decay chart: accuracy/F1 trend (Plotly line chart)
- Build alert history table with severity colour coding
- Add feature selector to deep-dive into distribution comparison (reference vs current)

**Week 9
Retraining
Trigger Logic**

- Build automatic retraining flag: set `retraining_needed=True` in DB when drift is CRITICAL
- Build a mock retraining job: retrain model on latest data, evaluate, promote if better
- Implement model version promotion logic: new model replaces baseline if metrics improve
- Add retraining history log to dashboard

**Week 10 Testing,
Hardening &
Docs**

- Write unit tests for all statistical test implementations
- Integration test: full pipeline from raw batch → drift score → alert
- Stress test: 1000 drift-check runs, measure throughput and DB write latency
- Write comprehensive README: architecture diagram, setup guide, config reference
- Record architecture walkthrough video for portfolio

■ Phase 2 — Future Enhancements

- ◆ Multi-model monitoring: track multiple deployed models from a single dashboard
 - ◆ SHAP-based drift root cause analysis: which feature value ranges are driving drift
 - ◆ Prediction confidence drift: monitor model uncertainty distribution over time
 - ◆ A/B model comparison: monitor champion vs challenger model performance in parallel
 - ◆ Integration with Prometheus + Grafana for production-grade observability stack
 - ◆ Online learning integration: models that auto-update on incoming data streams
-

■ Potential Challenges & Solutions

Challenge: Reference Window Selection

Solution: PSI and KS-test results depend heavily on the reference dataset size — use at least 1000 samples in the reference window; document the sensitivity of each test to sample size

Challenge: Labelled Data Delay

Solution: Concept drift requires ground truth labels, which arrive with a delay in production — design a label ingestion pipeline with configurable lag tolerance (e.g. 7-day label delay)

Challenge: Alert Fatigue

Solution: Naive alerting fires constantly for minor fluctuations — implement severity tiers (OK/WARNING/CRITICAL), cooldown periods, and aggregate-only-critical alerts to reduce noise

Challenge: Airflow Complexity

Solution: Airflow's learning curve is steep; start with Prefect (simpler Python-native API) and migrate to Airflow once the DAG logic is proven stable

Challenge: Statistical Test Interpretation

Solution: KS-test p-values are affected by sample size — with large batches, even tiny real-world-irrelevant differences become statistically significant; prefer effect sizes (PSI, JSD) over raw p-values

■ Learning Resources

- ◆ Evidently AI documentation — open-source model monitoring
- ◆ `scipy.stats` KS-test and Chi-Square documentation

- ◆ PSI (Population Stability Index) — Credit risk monitoring literature
 - ◆ Apache Airflow documentation — DAG authoring
 - ◆ MLflow Model Registry documentation
 - ◆ Chip Huyen: Designing ML Systems (Ch. 8 — Data Distribution Shifts)
-

■ Project Completion Criteria

- Baseline model trained and logged to MLflow with reference statistics
- Drift simulator generates synthetic covariate, prior, and concept drift batches
- KS-test, Chi-Square, PSI, and JSD all implemented and validated
- Page-Hinkley sequential drift test implemented for concept drift
- Drift scores logged to PostgreSQL after each pipeline run
- Slack and email alerting functional with severity tiers
- Airflow (or Prefect) daily DAG running end-to-end
- Retraining trigger flag logic implemented
- Streamlit monitoring dashboard showing all drift timelines and alert history
- Unit + integration tests passing, README and architecture diagram complete